

Universidad Tecnológica de la Mixteca

Instituto de Computación



Algoritmo de Primeros y Siguientes

Informe de Actividades — Equipo 1

INTEGRANTES DEL EQUIPO

- Lujan De la Rosa Cristian Emir
- Cortez Vega Jaime Fabian
- Carrasco Pacheco Levi Josue
- Nicolas Vasquez Joel Osmar
- Vasquez Moreno Zair De Jesus
- Arias Mendoza Angel Adrian

Prof. David Martínez Torres

Compiladores • 502-A • Octubre 2026

Modelo del programa principal

El sistema se estructura en tres capas bien diferenciadas que separan las responsabilidades de cada módulo y facilitan el mantenimiento y la extensibilidad del proyecto.

- **Capa de modelos.** Contiene la clase **Gramatica**, que representa de forma abstracta una gramática libre de contexto: almacena los conjuntos de terminales y no terminales, el símbolo inicial y el diccionario de producciones.
- **Capa de persistencia.** Agrupa tres módulos independientes: `lector_archivo.py`, encargado de parsear el archivo `.txt` y construir el objeto **Gramatica**; `algoritmo_primeros.py`, que calcula los conjuntos PRIMERO mediante iteración de punto fijo; y `algoritmo_siguietes.py`, que calcula los conjuntos SIGUIENTE con la misma estrategia.
- **Capa de presentación.** Contiene los módulos de interfaz gráfica contruidos con Tkinter: `estilos.py` define la paleta visual; `tabla_base.py` y `tabla_conjuntos.py` implementan los widgets de tabla reutilizables; `ventana_primeros_siguietes.py` presenta los resultados en pestañas; e `interfaz.py` actúa como punto de entrada de la aplicación.

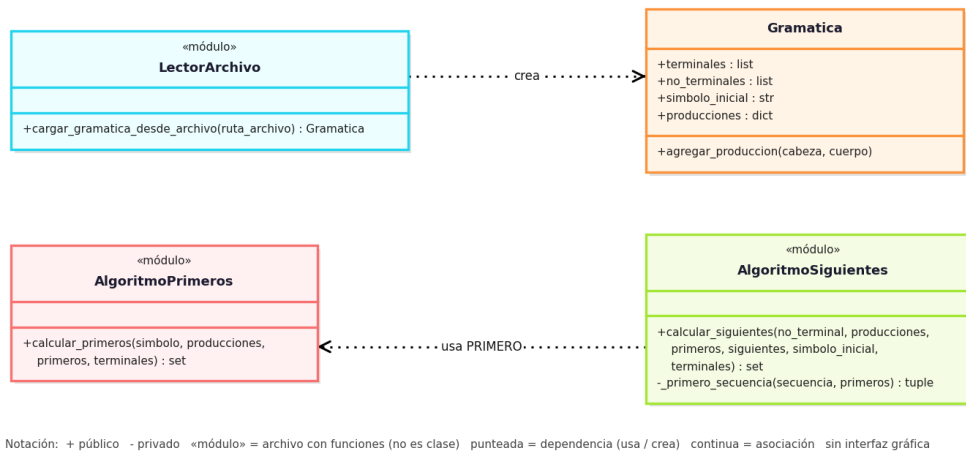


Figura 1: Modelo del programa principal — arquitectura en capas del sistema.

Análisis

Los conjuntos **PRIMERO** y **SIGUIENTE** constituyen estructuras formales fundamentales en el diseño de analizadores sintácticos predictivos (LL(1)). Su cómputo correcto a partir de una gramática libre de contexto determina, respectivamente, qué terminal puede iniciar una derivación (**PRIMERO**) y qué terminal puede aparecer inmediatamente a la derecha de un no terminal en alguna forma sentencial (**SIGUIENTE**).

Para el cálculo de ambos conjuntos se adoptó un enfoque iterativo de *punto fijo*: las reglas de inferencia se aplican de manera repetida hasta que ningún conjunto varía entre dos iteraciones consecutivas. Este método garantiza la convergencia para cualquier gramática libre de contexto finita.

Reglas de cómputo de PRIMERO

1. Si X es un símbolo terminal, entonces $\text{PRIMERO}(X) = \{X\}$.
2. Si existe la producción $X \rightarrow \varepsilon$, se incorpora ε a $\text{PRIMERO}(X)$.
3. Para cada producción $X \rightarrow Y_1 Y_2 \cdots Y_n$:
 - Se incorpora $\text{PRIMERO}(Y_1) - \{\varepsilon\}$ a $\text{PRIMERO}(X)$.
 - Si $\varepsilon \in \text{PRIMERO}(Y_i)$ para todo $i < k$, se incorpora también $\text{PRIMERO}(Y_k) - \{\varepsilon\}$.
 - Si $\varepsilon \in \text{PRIMERO}(Y_i)$ para todo $i \in \{1, \dots, n\}$, se incorpora ε a $\text{PRIMERO}(X)$.

Reglas de cómputo de SIGUIENTE

1. Se incorpora $\$$ a $\text{SIGUIENTE}(S)$, donde S denota el símbolo inicial de la gramática.
2. Para cada producción $A \rightarrow \alpha B \beta$:
 - Se incorpora $\text{PRIMERO}(\beta) - \{\varepsilon\}$ a $\text{SIGUIENTE}(B)$.
 - Si $\varepsilon \in \text{PRIMERO}(\beta)$ (o bien $\beta = \varepsilon$), se incorpora $\text{SIGUIENTE}(A)$ a $\text{SIGUIENTE}(B)$.

El sistema implementa ambos algoritmos en Python mediante una arquitectura modular organizada en tres capas: *modelos*, *persistencia* y *presentación*. La capa de modelos define la clase `Gramatica`; la capa de persistencia agrupa los módulos de lectura de archivos y de cómputo de los conjuntos; y la capa de presentación contiene la interfaz gráfica construida con Tkinter.

Asignación de actividades

Sub equipos:

1. Cortez Vega Jaime Fabian y Lujan De la Rosa Cristian Emir
2. Arias Mendoza Angel Adrian y Vasquez Moreno Zair De Jesus
3. Carrasco Pacheco Levi Josue y Nicolas Vasquez Joel Osmar

Equipo	Actividad	Entregas clave	Fecha de Entrega
Todos	Diseño de la estructura general del proyecto, definición de la clase <code>Gramatica</code> y comprensión del algoritmo de punto fijo.		Semana 1.
1	Implementación del algoritmo de PRIMERO y del algoritmo de SIGUIENTE mediante iteración de punto fijo.	Módulos <code>algoritmo_primeros.py</code> y <code>algoritmo_siguientes</code> funcionales con pruebas sobre gramáticas de ejemplo.	Semana 2.
2	Implementación de la clase <code>Gramatica</code> y del lector de archivos <code>.txt</code> en sus dos formatos (simple y con encabezados).	Módulos <code>gramatica.py</code> y <code>lector_archivo.py</code> con manejo de errores y soporte para ε .	Semana 2.
3	Diseño e implementación de la interfaz gráfica: ventana principal, editor de gramática con numeración de líneas y ventana de resultados con pestañas.	Módulos de presentación integrados en <code>interfaz.py</code> ; interfaz funcional y probada en Windows y macOS.	Semana 3.

- Es importante que cada integrante del equipo registre su avance diario en el informe de actividades, describiendo las tareas realizadas o los errores encontrados.
- La primera reunión de control de avances tendrá lugar al finalizar la semana 1, donde los sub equipos presentarán su diseño y resolverán dudas.
- La segunda reunión se realizará al cierre de la semana 2 para integrar los módulos de algoritmos y modelo de datos con la interfaz en desarrollo.
- Los días restantes se dedicarán a pruebas de integración, corrección de errores y presentación final con el profesor.

*Clases y métodos implementados por cada subequipo***Cortez Vega y Lujan De la Rosa:**

1. **Módulo Algoritmo PRIMERO** (`algoritmo_primeros.py`): Implementa la función `calcular_primeros`, que calcula los conjuntos PRIMERO para todos los símbolos de la gramática mediante un algoritmo iterativo de punto fijo. La función recibe el símbolo de consulta, el diccionario de producciones, un diccionario mutable de conjuntos `primeros` y la lista de terminales. Inicializa cada terminal con su propio conjunto y cada no terminal con el conjunto vacío; luego itera hasta que no haya cambios.
2. **Módulo Algoritmo SIGUIENTE** (`algoritmo_siguientes.py`): Implementa dos funciones:
 - a. **`_primero_secuencia`**: función auxiliar que calcula PRIMERO de una secuencia de símbolos e indica si toda la secuencia puede derivar en ε .
 - b. **`calcular_siguientes`**: función principal que aplica el algoritmo de SIGUIENTE mediante iteración de punto fijo. Inicializa el símbolo inicial con $\{\$ \}$ y, para cada producción $A \rightarrow \alpha B \beta$, propaga `PRIMERO( $\beta$ )` y, cuando corresponde, `SIGUIENTE( $A$ )` hacia `SIGUIENTE( $B$ )`.

Arias Mendoza y Vasquez Moreno:

1. **Módulo Gramática** (`gramatica.py`): Define la clase `Gramatica`, que representa una gramática libre de contexto. Sus atributos son: `terminales`, `no_terminales`, `simbolo_inicial` y `producciones` (diccionario de cabeza a lista de cuerpos). El método `agregar_produccion` inserta una nueva regla de producción al diccionario.
2. **Módulo Lector de Archivo** (`lector_archivo.py`): Implementa la función `cargar_gramatica_desde_archivo`, que lee un archivo `.txt` codificado en UTF-8 y construye un objeto `Gramatica`. Soporta dos formatos:
 - a. **Formato simple**: solo producciones; terminales y no terminales se infieren automáticamente cualquier símbolo que encabece una producción es no terminal, el resto son terminales.
 - b. **Formato con encabezados**: las dos primeras líneas declaran explícitamente los terminales y no terminales; las producciones inician en la tercera línea.

Acepta producciones vacías escritas como ε , `e` o `epsilon` (sin distinción de mayúsculas) y lanza `ValueError` ante cualquier producción malformada.

Carrasco Pacheco y Nicolas Vasquez:

1. **Módulo de Estilos** (`estilos.py`): Define la paleta de colores (`COLORES`), la función `configurar_estilos` que aplica los temas `ttk` globalmente, y la función `crear_superficie` que genera marcos con un diseño visual uniforme para todas las secciones de la interfaz.
2. **Módulo Tabla Base** (`tabla_base.py`): Widget `Treeview` reutilizable que incluye barras de desplazamiento vertical y horizontal. Permite configurar las columnas, sus encabezados, anchos y alineación, y es utilizado como base por los demás módulos de presentación.
3. **Módulo Tabla Conjuntos** (`tabla_conjuntos.py`): Especializa la tabla base para mostrar los conjuntos PRIMERO o SIGUIENTE, presentando un no terminal por fila y su conjunto correspondiente como cadena de símbolos separados por comas.
4. **Módulo Ventana de Resultados** (`ventana_primeros_siguientes.py`): Implementa la clase `VentanaPrimerosSiguientes`, una ventana secundaria con dos pestañas: una para los conjuntos PRIMERO y otra para los conjuntos SIGUIENTE. Cada pestaña muestra una `TablaConjuntos` con los resultados del análisis.
5. **Módulo Interfaz Principal** (`interfaz.py`): Implementa la clase `InterfazCompilador`, punto de entrada de la aplicación. Proporciona:
 - Carga de archivos `.txt` mediante diálogo del sistema operativo.
 - Editor de solo lectura con numeración de líneas para inspeccionar la gramática.
 - Botón **Analizar** que invoca los módulos de PRIMERO y SIGUIENTE y abre la ventana de resultados.
 - Botón **Limpiar** para reiniciar el estado de la aplicación.
 - Barra de estado con mensajes informativos sobre el progreso del análisis.

Avances

Fecha	Lujan De la Rosa Cristian Emir	Cortez Vega Jaime Fabian	Arias Mendoza Angel Adrian
28/09 (Lun)	Organización del sub equipo y revisión del algoritmo de PRIMERO.	Organización del sub equipo y revisión del algoritmo de SIGUIENTE.	Organización del sub equipo y diseño de la clase Gramatica .
29/09 (Mar)	Diseño de la función calcular_primeros y su estructura de datos.	Diseño de la función auxiliar _primero_secuencia .	Implementación de la clase Gramatica con atributos y método agregar_produccion .
30/09 (Mié)	Implementación del ciclo de punto fijo para el cálculo de PRIMERO.	Implementación de calcular_siguientes con inicialización de \$.	Inicio del módulo lector_archivo.py para el formato simple de gramáticas.
01/10 (Jue)	Pruebas con gramáticas simples y corrección del manejo de ε .	Pruebas del algoritmo y ajuste de la propagación de SIGUIENTE(A).	Implementación del soporte para el formato con encabezados en el lector.
02/10 (Vie)	Entrega del módulo algoritmo_primeros.py y apoyo en SIGUIENTE.	Entrega del módulo algoritmo_siguientes.py al equipo.	Manejo de errores: producciones vacías (ε , e, epsilon) y no terminales no declarados.
05/10 (Lun)	Revisión de errores reportados por el sub equipo de interfaz.	Pruebas con gramáticas del directorio ArchivosdePrueba .	Pruebas del lector con los seis archivos de gramática de prueba.
06/10 (Mar)	Pruebas de integración del algoritmo PRIMERO con el lector de archivos.	Corrección del cálculo de SIGUIENTE para gramáticas con recursión izquierda.	Corrección de la detección automática de terminales y no terminales.
07/10 (Mié)	Ajustes al manejo de gramáticas con múltiples producciones vacías.	Integración final y pruebas cruzadas con el sub equipo de modelo.	Integración con los módulos de algoritmos y pruebas de carga.
08/10 (Jue)	Validación con los seis archivos de gramática de prueba.	Documentación de los módulos algoritmo_primeros.py y algoritmo_siguientes.py	Ajustes finales al lector y revisión de errores de codificación UTF-8.
09/10 (Vie)	Revisión del informe y entrega del proyecto.	Entrega del proyecto y presentación al profesor.	Revisión del informe y entrega del proyecto.

Fecha	Vasquez Moreno Zair De Jesus	Nicolas Vasquez Joel Osmar	Carrasco Pacheco Levi Josue
28/09 (Lun)	Organización del sub equipo y análisis de los formatos de gramática.	Estudio de Tkinter y diseño de la arquitectura de la interfaz gráfica.	Estudio de la interfaz propuesta y planificación de la ventana principal.
29/09 (Mar)	Apoyo en el diseño del modelo Gramatica y pruebas unitarias.	Implementación de estilos.py con paleta de colores y temas ttk.	Implementación del editor de gramática con numeración de líneas sincronizada.
30/09 (Mié)	Implementación del soporte para separadores -> y en el lector.	Desarrollo del widget tabla_base.py con Treeview y scrollbars.	Desarrollo de los botones de control: Cargar, Analizar y Limpiar.
01/10 (Jue)	Pruebas del lector con gramáticas simples y con encabezados.	Implementación de tabla_conjuntos.py para mostrar PRIMERO y SIGUIENTE.	Implementación del diálogo de selección de archivos y barra de estado.
02/10 (Vie)	Validación del símbolo inicial y detección de no terminales faltantes.	Desarrollo de ventana_primeros_sigue con pestañas de resultados.	Integración de la ventana principal con el módulo lector de archivos.
05/10 (Lun)	Reporte de errores al sub equipo 1 tras las primeras pruebas de integración.	Integración de la ventana de resultados con los módulos de algoritmos.	Pruebas de la interfaz completa y corrección de errores visuales.
06/10 (Mar)	Corrección del manejo de líneas en blanco y codificación UTF-8 en el lector.	Pruebas de la interfaz con gramáticas de ejemplo y corrección de errores.	Ajuste de la sincronización del scroll entre editor y numeración de líneas.
07/10 (Mié)	Integración con la interfaz y pruebas de carga de archivos desde la GUI.	Corrección de la visualización de conjuntos con símbolos especiales.	Pruebas de integración completa del sistema con todos los módulos.
08/10 (Jue)	Revisión final del módulo lector y su documentación.	Ajustes finales a los estilos y empaquetado del ejecutable Windows.	Configuración de PyInstaller para el ejecutable .exe en Windows.
09/10 (Vie)	Entrega del proyecto y apoyo en la presentación.	Revisión final y entrega del proyecto.	Entrega final del proyecto y presentación al profesor.

Conclusiones

El desarrollo del presente proyecto permitió al equipo comprender de manera rigurosa y práctica el papel que desempeñan los conjuntos **PRIMERO** y **SIGUIENTE** en la construcción de analizadores sintácticos predictivos. A diferencia del análisis léxico que reconoce unidades léxicas mediante autómatas finitos deterministas el análisis sintáctico predictivo exige razonar sobre las derivaciones posibles de la gramática; los conjuntos **PRIMERO** y **SIGUIENTE** constituyen la herramienta formal que hace posible tomar tales decisiones de manera determinista y sin necesidad de retroceso.

La implementación del algoritmo iterativo de punto fijo exigió un análisis cuidadoso de las dependencias entre los conjuntos de distintos no terminales: una modificación en $\text{PRIMERO}(X)$ puede propagarse hacia los conjuntos **SIGUIENTE** de otros símbolos en cadena. El manejo correcto de dichas dependencias mediante la repetición controlada de las reglas de inferencia representó el núcleo algorítmico del proyecto y reforzó la comprensión teórica de la convergencia de procesos iterativos de punto fijo.

Desde la perspectiva del diseño de software, la separación en tres capas modelos, persistencia y presentación facilitó la distribución del trabajo entre los sub equipos y permitió desarrollar y verificar cada módulo de forma independiente antes de la fase de integración. La interfaz gráfica construida con Tkinter añadió valor práctico al sistema al hacer accesible el cálculo de los conjuntos a cualquier usuario, sin requerir conocimiento del código fuente.

En síntesis, el proyecto consolidó la comprensión de la segunda fase en el diseño de compiladores y reforzó buenas prácticas de ingeniería de software modularidad, pruebas incrementales y documentación que serán aplicadas en las etapas subsecuentes del curso.

Referencias

Martínez Torres, D. (s. f.). *Tema 3. Análisis sintáctico* [Diapositivas de PowerPoint]. Instituto de Computación, Universidad Tecnológica de la Mixteca.

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.1 ed.). Pearson Educación.

Mogensen, T. Æ. (2017). *Introduction to Compiler Design* (2nd ed.). Springer.